HOME CARE FLOW
TECNOLOGIA E CUIDADO EM HARMONIA

Guia da Apresentação do Projeto Final

Como o Home Care Flow atende, item por item, ao que foi pedido no documento "Projeto Final — Aranduê", com as falas e o roteiro da demonstração.

ODS 8 · ODS 10 | MySQL | Node.js | ReactJS | CRUD completo ×3 | Publicado na nuvem

Autor: [seu nome completo] · GitHub @ClaudeirZ2
Formação: Recode Pro AI — Projeto Final Aranduê
Sistema: https://homecare-flow.vercel.app · **Código:** github.com/ClaudeirZ2/homecare_flow

COLA RÁPIDA

Tudo o que você precisa em uma página

Leia isto 5 minutos antes de entrar. O resto do guia explica cada item com calma.

Links
Site: https://homecare-flow.vercel.app
API: https://homecare-flow.onrender.com
Documentação da API: .../api/docs

Logins (senha `123456`)
Admin: admin@homecareflow.com
Gestor: carlos@homecareflow.com
Técnica: juliana@homecareflow.com

QR Code para o check-in: `QR-PAC-0001`

Checklist antes de começar

- ☐ Abrir o link da API **5 min antes** (o servidor grátis "acorda" em até 1 min)
- ☐ Site aberto na tela de login, com zoom de 110–125% para o projetor
- ☐ Celular com o app instalado (opcional, para o bônus)
- ☐ Plano B: as telas em `docs/telas` ou um vídeo
- ☐ Respirar: ninguém conhece o sistema melhor que você

Roteiro em 10 minutos

Tempo	Bloco	Mensagem principal	Item do edital
1 min 30	1. Abertura e problema	O ponto em papel e o pagamento pelo WhatsApp prejudicam quem cuida.	Seção 5: problema
1 min 30	2. ODS, público e impacto	ODS 8: trabalho formal, horas comprovadas e pagamento justo.	Seções 1 e 5
5 min	3. Demonstração ao vivo	CRUD de Escalas, financeiro e o dia da técnica (check-in, prontuário e ganhos).	Seção 4 (CRUD integrado)
1 min 30	4. Como foi construído	MySQL (8 tabelas), API Node em camadas, React componentizado e nuvem.	Seções 3 e 4
30 s	5. Encerramento	Tecnologia simples dando dignidade e transparência para quem cuida.	Seção 6: impacto

As 3 frases que a banca precisa lembrar

1 · Problema real

Cuidadores não conseguem comprovar as horas que trabalham.

2 · Solução completa

Da escala ao pagamento: check-in com GPS/QR, prontuário e cálculo automático.

3 · Impacto (ODS 8)

O profissional vê, no celular, quanto tem a receber pelas horas reais.

VISÃO GERAL

O que o edital pede × o que o projeto entrega

Use esta tabela para mostrar, se perguntarem, que nenhum requisito ficou de fora.

Item do documento	O que foi pedido	Como o Home Care Flow atende	
1. Tema e ODS	Alinhar a pelo menos uma ODS (1, 5, 8, 10 ou 17)	**ODS 8** (principal) e **ODS 10** (complementar)	✓
2. Restrições	Nada para menores de 18; nada de esporte, saúde ou religião	Tema **autorizado pelos gestores**, com foco na gestão do trabalho. Pacientes: só 18+, validado na API.	✓
3. Tecnologias	MySQL, Node.js (API REST) e ReactJS	MySQL 8, Node.js + Express e React (Vite)	✓
4.1 Banco de dados	Tabelas bem definidas e relacionadas	**8 tabelas** com chaves estrangeiras, índices e transações	✓
4.2 Backend	API RESTful com **pelo menos 1 CRUD completo integrado ao front**	**3 CRUDs completos** (Escalas, Pacientes e Cuidadores), todos usados pela interface	✓
4.3 Frontend	React componentizado: Navbar, Cards, Formulários, Botões, Tabelas, Modais, Footer	Todos os 7 componentes existem e são reutilizados em várias telas	✓
5. Apresentação	Explicar problema, ODS, público-alvo e impacto	Blocos 1 e 2 do roteiro, mais o README do projeto	✓
6. Critérios	Organização, funcionamento, código, clareza, criatividade, impacto, integração	Ver a página "Critérios de avaliação" deste guia	✓

Além do mínimo pedido (diferenciais)

Funcionalidades

- Check-in por **QR Code lido pela câmera** ou por **GPS com cálculo da distância**
- **Meus ganhos**: o técnico vê quanto tem a receber
- Demonstrativo de pagamento em PDF e exportação para Excel
- Alertas automáticos e **auditoria** (quem fez o quê)

Qualidade técnica

- **3 perfis de acesso**, com regras na API (39 verificações)
- Documentação da API no **Swagger** e testes automatizados
- **PWA**: instala no celular e funciona com internet ruim
- Publicado na nuvem: Aiven, Render e Vercel

EDITAL · SEÇÕES 1 E 2

Tema do projeto, ODS e restrições

O edital exige alinhamento a uma ODS e lista restrições de tema.

O QUE FOI PEDIDO

"O projeto deve estar obrigatoriamente alinhado a pelo menos um dos ODS": 1, 5, 8, 10 ou 17.

COMO O PROJETO ATENDE

ODS 8, Trabalho Decente e Crescimento Econômico (principal). O sistema formaliza a jornada dos cuidadores: o check-in e o check-out são registrados com GPS ou com o QR Code da casa do paciente, e o pagamento é calculado automaticamente sobre as horas reais, com status pendente/pago.

ODS 10, Redução das Desigualdades (complementar). Dá transparência e previsibilidade de renda para uma categoria em grande parte informal.

O QUE FALAR

"O projeto está alinhado principalmente à ODS 8. Ele formaliza o trabalho de quem cuida: a entrada e a saída ficam registradas, e o pagamento é calculado pelas horas reais. Também conversa com a ODS 10, porque é uma categoria muito informal, e ter as horas registradas dá segurança de renda."

Restrições do edital

O QUE FOI PEDIDO

Não fazer projetos voltados a **menores de 18 anos**, nem relacionados a **esporte, saúde ou religião**.

COMO O PROJETO ATENDE

- **Saúde:** o tema foi **autorizado pelos gestores do curso**. O foco é a **gestão do trabalho** (escalas, jornada e pagamento), e não diagnóstico ou tratamento.
- **Menores de 18:** o público são profissionais adultos, e o sistema **não aceita pacientes com menos de 18 anos**. A regra está na API e na tela.

SE PERGUNTAREM

"Eu pedi autorização para esse tema e ela foi concedida. O foco não é a saúde em si; é a gestão do trabalho de quem cuida: a escala, a jornada e o pagamento. Por isso a ODS escolhida é a 8."

Fale da autorização com tranquilidade e sem se justificar demais: ela é um fato, não um problema.

EDITAL · SEÇÕES 3 E 4.1

Tecnologias obrigatórias e banco de dados MySQL

"O projeto deve obrigatoriamente utilizar MySQL, Node.js (API RESTful) e ReactJS."

ReactJS
Interface componentizada
(Vercel)

→

Node.js + Express
API REST com JWT
(Render)

→

MySQL 8
8 tabelas relacionadas
(Aiven)

O QUE FOI PEDIDO (4.1)

"Banco de dados estruturado em MySQL, contendo tabelas bem definidas e relacionadas ao funcionamento completo da aplicação."

Tabela	Para que serve	Relacionamentos
`usuarios`	Login (admin, gestor, técnico) com senha criptografada	1:1 com profissionais; 1:N com auditoria
`profissionais`	Especialidade, valor da hora e disponibilidade	1:N com escalas e com pagamentos
`pacientes`	Quem recebe o atendimento, o código do QR e a localização da casa	1:N com escalas
`escalas`	Planejamento: paciente, profissional, data e turno	1:1 com plantões
`plantoes`	Execução real: horários, local e distância do check-in	1:N com prontuários
`prontuarios`	Registro padronizado de cada plantão	N:1 com plantões
`pagamentos`	Fechamento mensal (único por profissional e mês)	N:1 com profissionais
`auditoria`	Quem fez o quê e quando (LGPD)	N:1 com usuários

PONTOS FORTES PARA CITAR

- **Chaves estrangeiras** em todos os relacionamentos, mais restrições únicas (ex.: uma escala gera no máximo um plantão).
- **Transações:** o check-in muda 3 tabelas de uma vez. Se uma etapa falha, nada é gravado (rollback).
- **Exclusão lógica** de pacientes e profissionais, para não perder o histórico.
- `schema.sql` e `seed.sql` recriam o banco com um comando, com dados de demonstração realistas.

O QUE FALAR

"O banco tem 8 tabelas relacionadas. Uso chaves estrangeiras e transações: no check-in, três tabelas mudam juntas, e se uma falhar nada é gravado. Pacientes e profissionais nunca são apagados de verdade, só desativados, para não perder o histórico."

EDITAL · SEÇÃO 4.2

Backend: API RESTful em Node.js e CRUD completo

Este é o requisito mais cobrado: o CRUD precisa funcionar **100% pela interface**.

O QUE FOI PEDIDO

"A aplicação deve possuir pelo menos **1 CRUD completo**, acessível e integrado ao frontend": criar, listar, atualizar e remover, "todas as operações funcionando diretamente através da interface gráfica".

COMO O PROJETO ATENDE: 3 CRUDS COMPLETOS

Operação	Escalas	Pacientes	Cuidadores	Na tela
Create	`POST /api/escalas`	`POST /api/pacientes`	`POST /api/profissionais`	Botão "Novo..." + formulário
Read	`GET /api/escalas`	`GET /api/pacientes`	`GET /api/profissionais`	Tabela/cards, filtros e busca
Update	`PUT /api/escalas/:id`	`PUT /api/pacientes/:id`	`PUT /api/profissionais/:id`	Ícone de lápis + formulário
Delete	`DELETE /api/escalas/:id`	`DELETE /api/pacientes/:id`	`DELETE /api/profissionais/:id`	Lixeira + modal de confirmação

Como a API está organizada (camadas)

routes define caminho e método → **middlewares** login (JWT) e perfil → **controllers** validações e regras → **models** consultas SQL

Segurança

- Senhas com **bcrypt** e login com **JWT** (expira em 8h)
- Limite de tentativas de login e cabeçalhos **helmet**
- SQL sempre parametrizado (contra SQL Injection)
- Erros padronizados: 400, 401, 403, 404 e 409

Regras de negócio

- Não deixa o mesmo profissional em duas escalas no mesmo turno
- Check-in só no dia da escala e só pelo próprio técnico
- Pagamento = minutos trabalhados ÷ 60 × valor da hora
- Documentação no **Swagger** (`/api/docs`) e **testes automatizados**

O QUE FALAR

"A API é REST e está organizada em camadas: rotas, middlewares de login e permissão, controllers com as validações e models com o SQL. O edital pede um CRUD completo; eu fiz três: Escalas, Pacientes e Cuidadores, todos funcionando pela interface. Vou mostrar o de Escalas ao vivo."

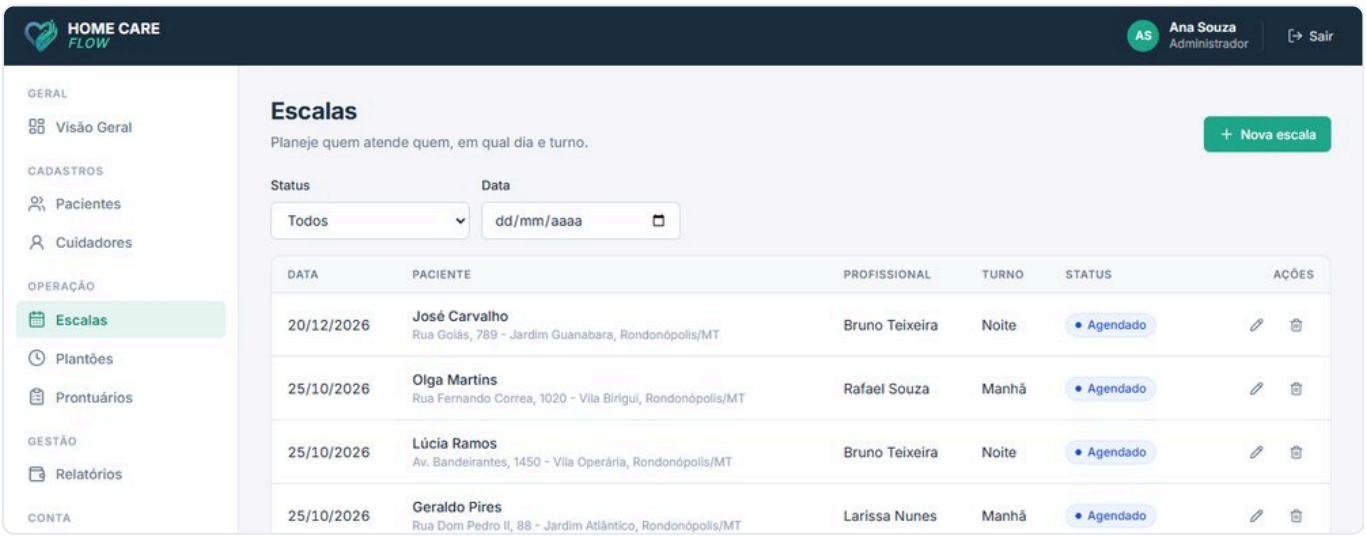
EDITAL · SEÇÃO 4.3

Frontend: ReactJS componentizado

"Seguir rigorosamente o conceito de componentização, evitando páginas construídas em arquivo único."

Exemplo do edital	Componente no projeto	Onde aparece
Navbar	`layout/Navbar`, `Sidebar`, `Layout`, `Logo`	Todas as telas
Cards	`ui/Card`, `CardPaciente`, `PainelTecnico`	Painel, Pacientes, Financeiro
Formulários	`ui/Input`, `FormularioEscala`, `FormularioPaciente`, `FormularioProfissional`, `FormularioProntuario`	Cadastros e prontuário
Botões	`ui/Button` (variantes primary, outline, danger, ghost...)	O sistema inteiro
Tabelas	`ui/Table`, `TabelaEscalas`, `TabelaProfissionais`, `TabelaPagamentos`	Escalas, Cuidadores, Financeiro (viram cartões no celular)
Modais	`ui/Modal`, `ModalConfirmacao`, `ModalCheckinQr`, `ModalQrCode`, `ModalDemonstrativo`	Formulários, exclusões, QR e pagamento
Footer	`layout/Footer`	Todas as telas

Organização: `pages/` (telas) → `components/` (uma pasta por módulo, mais `ui/` e `layout/`) → `services/` (chamadas à API). O login e os avisos ficam em *contexts* (`AuthContext` e `ToastContext`).



Tela de Escalas: PageHeader, Button, filtros (Input), Table e Badge, todos reaproveitados nas outras telas

O QUE FALAR

"O front é todo componentizado. Navbar, cards, formulários, botões, tabelas, modais e footer são componentes que eu reaproveito em todas as telas: a mesma tabela serve para escalas, cuidadores e pagamentos, e o mesmo modal serve para editar e para confirmar exclusões."

EDITAL · SEÇÃO 5

O que a apresentação deve mostrar

O edital pede que fique claro: **problema, ODS, público-alvo e impacto**. Estas são as falas dos blocos 1 e 2 do roteiro.

💡 Problema

abertura · 1 min 30

"Boa tarde! Meu nome é [seu nome] e meu projeto é o Home Care Flow: tecnologia e cuidado em harmonia. Vocês já pensaram em como funciona o dia a dia de um cuidador que atende pacientes em casa? Em muitas equipes, o ponto é anotado num papel, o que dá margem para erro e até fraude. As anotações do plantão ficam num caderno, cada um escreve de um jeito. E o pagamento é combinado pelo WhatsApp: no fim do mês ninguém sabe ao certo quantas horas foram trabalhadas. Quem mais perde é o profissional, que não consegue comprovar o próprio trabalho."

🌎 ODS

30 s

"O projeto atende à ODS 8, Trabalho Decente. O sistema registra a entrada e a saída com GPS ou com o QR Code da casa do paciente, e calcula o pagamento automaticamente pelas horas reais. Também conversa com a ODS 10, porque dá segurança de renda a uma categoria muito informal."

👥 Público-alvo

20 s

"O público são os cuidadores e técnicos de enfermagem, todos adultos, e os gestores de pequenas agências de cuidado domiciliar."

💔 Impacto esperado

40 s

"O impacto que eu espero: fim do ponto em papel; um pagamento justo, que o próprio profissional consegue conferir no celular; um histórico de atendimento padronizado; e mais profissionalização para um setor que ainda é muito informal."

💡 Fale devagar e olhe para a banca, não para a tela. A pergunta do início ("vocês já pensaram...") prende a atenção.

ROTEIRO · BLOCO 3

Demonstração ao vivo (5 minutos)

Primeiro como **gestão** (o CRUD e o financeiro), depois como **técnica** (o dia de trabalho).

3.1 Login e perfis

30 s

Entrar com `admin@homecareflow.com` / `123456`

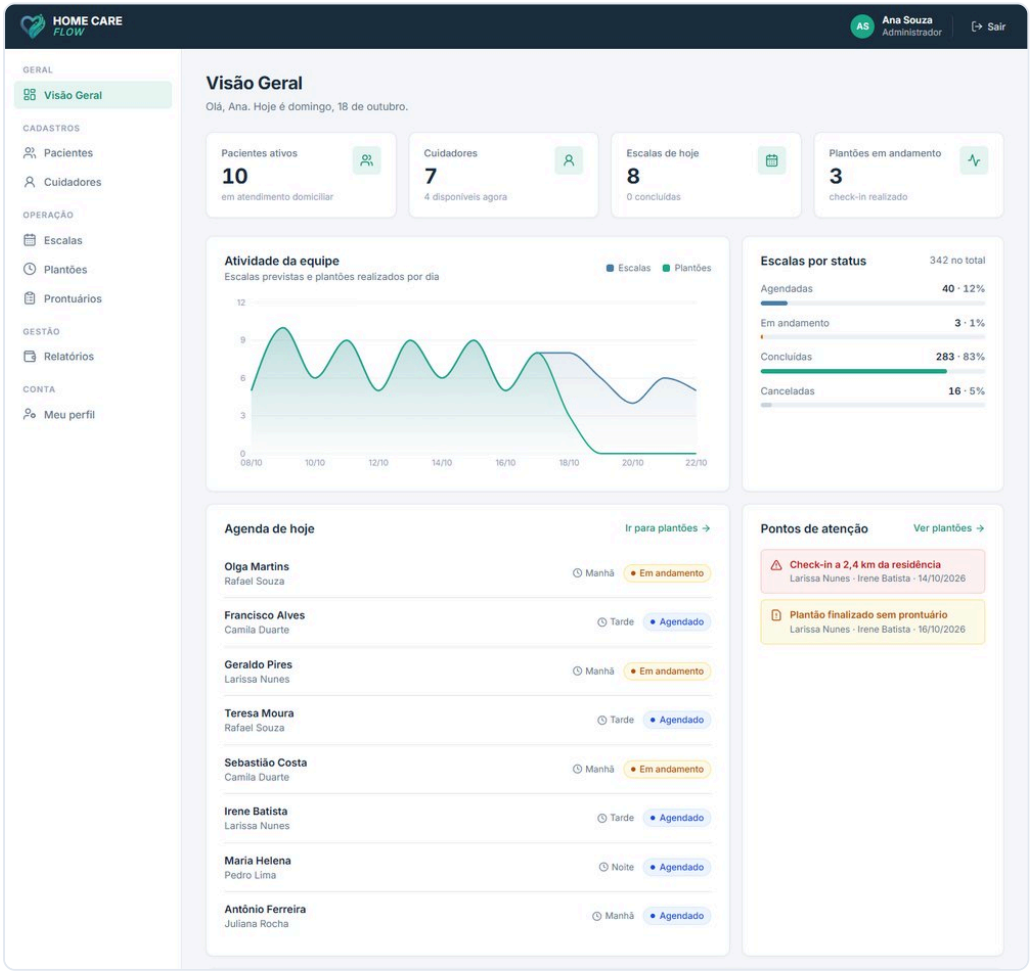
> *"O sistema tem três perfis, e cada um vê só o que precisa: o administrador vê tudo, o gestor cuida da operação sem excluir cadastros, e o técnico vê só o próprio trabalho."*

3.2 Visão Geral

50 s

Mostrar os cards, o gráfico, os "Pontos de atenção" e a "Atividade recente"

> *"Aqui é o painel da gestora: os indicadores do dia, o gráfico de escalas previstas versus plantões realizados e os pontos de atenção. O sistema avisa sozinho que um check-in foi feito a 2,4 km da casa do paciente e que um plantão terminou sem prontuário. Embaixo fica a auditoria: quem fez o quê e quando."*



Visão Geral (admin): indicadores, gráfico, agenda, alertas e auditoria

3.3 CRUD de Escalas ⭐ requisito obrigatório

1 min 30

1. **Criar:** "Nova escala" → paciente, profissional, data futura e turno → Criar. *"Aqui eu crio..."*
2. **Listar:** mostrar a escala na tabela e usar o filtro de status. *"...listo e filtro..."*
3. **Editar:** lápis → trocar o turno → Salvar. *"...edito..."*
4. **Regra:** criar outra igual (mesmo profissional, data e turno). *"...e o sistema não deixa duplicar."*
5. **Excluir:** lixeira → confirmar no modal. *"...e excluo, com confirmação."*

> *"Esse é o CRUD completo que o edital pede, funcionando todo pela tela. Pacientes e Cuidadores também têm CRUD completo."*

3.4 Cartão de QR Code

20 s

Pacientes → ícone de QR no card do Antônio Ferreira

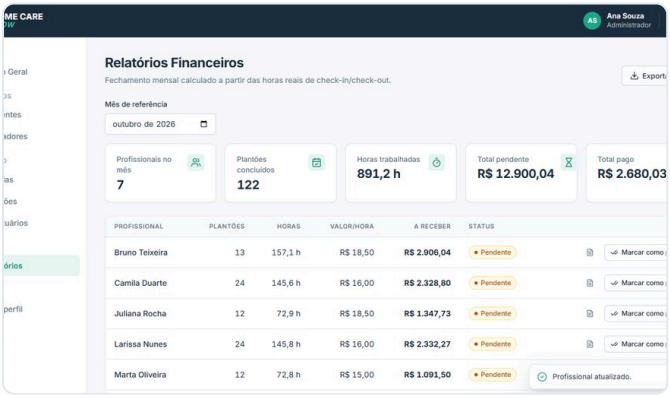
> *"Para cada paciente o sistema gera um cartão com QR Code para imprimir e colar na casa. O cuidador lê esse código com o celular ao chegar."*

3.5 Financeiro

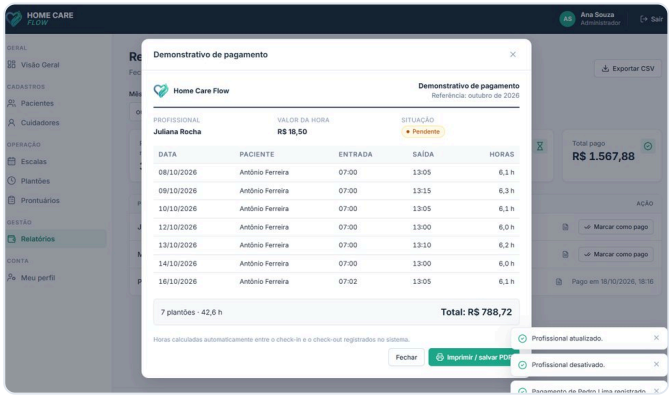
50 s

Relatórios → ícone de documento (demonstrativo) → Marcar como pago → Confirmar → Exportar CSV

> *"No fim do mês o sistema soma as horas reais de cada plantão e multiplica pelo valor da hora. Aqui está o demonstrativo, que pode ser impresso ou salvo em PDF. A gestora registra o pagamento e pode exportar para o Excel."*



Fechamento do mês



Demonstrativo de pagamento

3.6 O dia da técnica

1 min 30

Sair → entrar com `juliana@homecareflow.com` / `123456`

"Agora eu sou a Juliana, técnica de enfermagem. O menu dela é menor: ela não vê a lista de cuidadores nem o financeiro da empresa. Em 'Meus pacientes' aparecem só os pacientes que ela atende. O painel 'Meu dia' mostra o plantão de hoje e quanto ela já tem a receber."

Meus ganhos

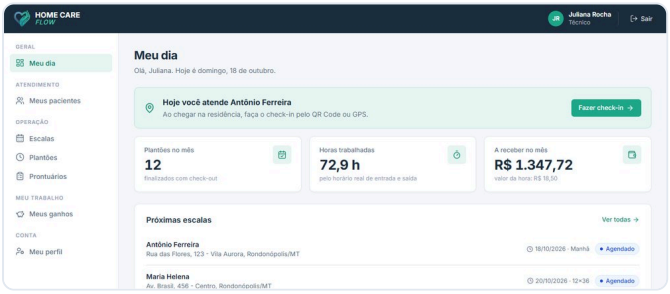
"Esse é o coração da ODS 8: ela acompanha as próprias horas e o valor, calculado automaticamente, sem papel e sem combinado por WhatsApp."

Plantões → QR Code → aba "Digitar código" → `QR-PAC-0001` → Confirmar

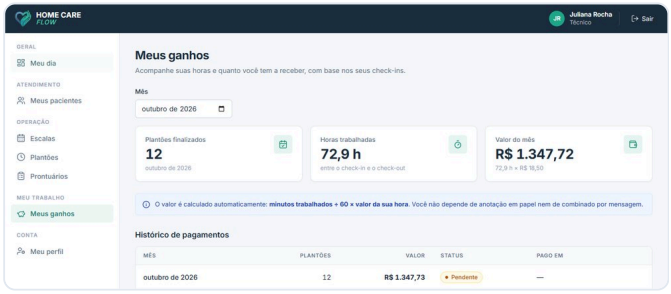
"Ao chegar na casa, ela faz o check-in lendo o QR Code com a câmera do celular. Se o código não for daquela casa, o sistema recusa."

Prontuários → plantão "em andamento" → pressão, temperatura e observação → Registrar → Plantões → Check-out

"Durante o plantão ela faz o registro padronizado, que já aparece no histórico do paciente. Na saída, o check-out, e essas horas já entram no fechamento do mês."



"Meu dia": o painel da técnica



"Meus ganhos": transparência para quem trabalha

3.7 Bônus: o app no celular

30 s

"Como o cuidador usa o celular na casa do paciente, o sistema é um PWA: dá para instalar como aplicativo, ele abre em tela cheia e funciona mesmo com internet ruim. No celular, as tabelas viram cartões fáceis de tocar."

💡 Para instalar antes: no Android, menu ☰ → "Instalar app"; no iPhone, Safari → Compartilhar → "Adicionar à Tela de Início".



ROTEIRO · BLOCOS 4 E 5

Como foi construído e encerramento

4. Parte técnica — 1 min 30

*"Usei as três tecnologias da formação. No **MySQL**, 8 tabelas relacionadas, com chaves estrangeiras e transações.*

*No **Node.js com Express**, uma API REST documentada com Swagger e organizada em camadas, com login por JWT, senhas com bcrypt, permissão por perfil, limite de tentativas de login e testes automatizados.*

*No **React**, tudo componentizado: navbar, cards, formulários, botões, tabelas, modais e footer reaproveitados em todas as telas.*

E está tudo publicado na nuvem: o banco no Aiven, a API no Render e o site na Vercel. Por isso estou usando um link, e não o meu computador."

5. Encerramento — 30 s

"Como próximos passos, quero transformar o sistema num aplicativo que funcione também sem internet, buscar a localização da casa pelo endereço e enviar lembretes do plantão. O Home Care Flow mostra que tecnologia simples, bem aplicada, pode dar dignidade e transparência para quem cuida. Obrigado! Fico à disposição para perguntas."

EDITAL · SEÇÃO 6

Critérios de avaliação: como o projeto responde a cada um

Critério	Evidência no projeto
Organização e arquitetura	Backend em camadas (routes, middlewares, controllers, models); front com pages, components e services; README e docs.
Funcionamento e fluidez	Publicado na nuvem; avisos de sucesso/erro; carregamento animado; funciona no celular (PWA).
Qualidade do código	Validações na API, SQL parametrizado, transações, testes automatizados e Swagger. Testado tela por tela.
Clareza da proposta	Problema concreto (ponto em papel, pagamento informal) e uma solução que cobre da escala ao pagamento.
Criatividade e originalidade	Check-in por QR Code da casa, GPS com distância, "Meus ganhos" e demonstrativo de pagamento.
Impacto social	ODS 8 e 10: horas comprovadas e renda previsível para uma categoria informal.
Integração front + back + banco	Cada ação da tela chama a API, que grava no MySQL; 3 CRUDs completos funcionando pela interface.

PREPARAÇÃO

Perguntas prováveis da banca

Respostas curtas. Fale com as suas palavras: o importante é mostrar que você entende o que fez.

"Por que um projeto de saúde, se o edital não permite?"

> *"Pedi autorização aos gestores e ela foi concedida. O foco é a gestão do trabalho de quem cuida (escala, jornada e pagamento), não o diagnóstico ou o tratamento. Por isso a ODS 8."*

"Onde está o CRUD completo?"

> *"Em Escalas, Pacientes e Cuidadores. As quatro operações funcionam pela tela e chamam a API: POST para criar, GET para listar, PUT para editar e DELETE para excluir."*

"Como funciona o login? O que é JWT?"

> *"A API confere a senha com o bcrypt e devolve um token JWT assinado. O front manda esse token em toda requisição, e a API sabe quem é o usuário e o perfil dele sem ir ao banco toda vez. Ele expira em 8 horas."*

"Como você garante que cada um vê só o que pode?"

> *"As regras estão na API, não só na tela. O técnico não vê os colegas nem pacientes que não atende, e a gestão não faz check-in no lugar dele. Testei 39 regras, perfil por perfil; quem tenta o que não pode recebe erro 403."*

"O que é uma transação e onde você usou?"

> *"É um bloco que acontece inteiro ou não acontece. No check-in eu crio o plantão e mudo o status da escala e do profissional. Se uma etapa falhar, faço rollback. Também uso no cadastro do profissional."*

"Como o GPS impede fraude?"

> *"A API calcula a distância até a casa do paciente (fórmula de Haversine). Acima de 300 metros vira um alerta para a gestão. E o QR Code prova a presença, porque o cartão fica colado na casa."*

"E a LGPD?"

> *"Senhas criptografadas, acesso por perfil, auditoria de quem fez cada ação e as senhas do servidor fora do GitHub. Todos os dados da demonstração são fictícios."*

"Por que não apagar de verdade o paciente ou o profissional?"

> *"Porque eles têm histórico de plantões, prontuários e pagamentos. Faço exclusão lógica: marco como inativo, cancelo as escalas futuras e bloqueio o login."*

"Tem testes? Tem documentação da API?"

> *"Tem. O npm test roda testes automatizados do backend, e testei todas as telas de ponta a ponta. A documentação está no Swagger, em /api/docs."*

"Você usou IA?"

> *"Usei como assistente de programação, para tirar dúvidas, revisar e acelerar partes repetitivas. As decisões do projeto, as regras de negócio e os testes são meus, e eu sei explicar cada parte."*

"Qual foi a maior dificuldade?"

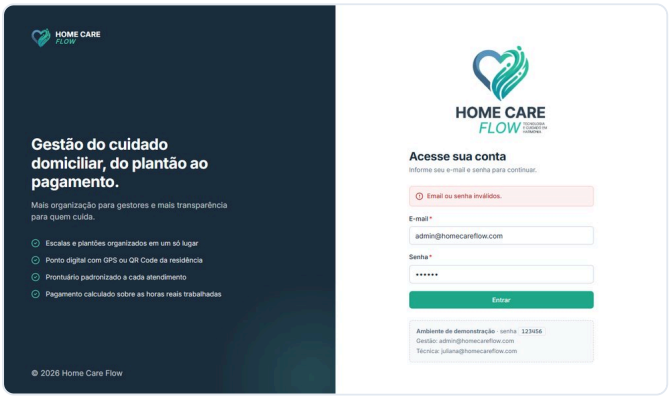
> *Escolha uma que você viveu: o **fuso horário** (datas com um dia a menos; na nuvem, o horário 3 horas adiantado), as **regras de permissão** por perfil ou a **publicação na nuvem** (SSL, CORS e as exigências do MySQL do Aiven).*

PLANO B

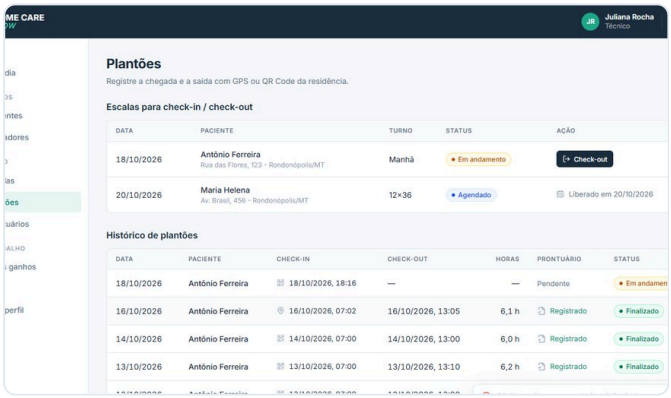
Se algo der errado na hora

Problema	O que fazer
Site não carrega ou aparece "Sem conexão com o servidor"	A API está acordando. Espere até 1 minuto e use o tempo para falar da arquitetura (bloco 4).
A internet do local caiu	Mostre as telas em `docs/telas` (ou o vídeo) e explique o fluxo com elas.
O navegador pede permissão de GPS ou câmera	Clique em "Permitir", ou use o QR Code pela aba "Digitar código" (não depende de GPS nem de câmera).
Os dados ficaram bagunçados depois de testes	Antes de sair de casa, rode `npm run db:setup:nuvem` na pasta backend: o banco volta para a demonstração.
Esqueceu uma fala	Volte para a tela e descreva o que ela mostra. A demonstração fala por você.
Pergunta que você não sabe responder	"Boa pergunta. Isso ainda não está no sistema, mas seria um ótimo próximo passo." Honestidade conta pontos.

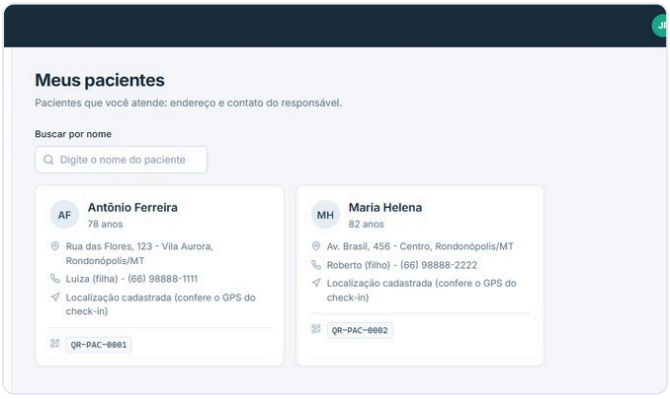
Telas de apoio



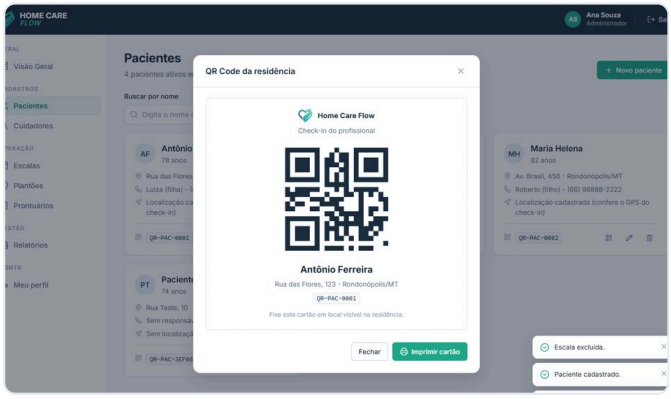
Login com os usuários de demonstração



Plantões: check-in por GPS ou QR Code



Técnica vê só os seus pacientes



Cartão de QR Code da residência